



Analyse Technique : Pipeline CI/CD (ci.yaml)

Ce document détaille la conception du pipeline d'Intégration et de Déploiement Continu (CI/CD) de **GeoMeteo**. Ce fichier est le "Gardien du Temple" : il automatise la validation du code et la livraison de l'application.

1. Déclencheurs & Stratégie de Branche

on:

push:

branches: ["main"]

pull_request:

branches: ["main"]



Analyse Technique :

- **Filtrage par branche** : Le pipeline ne se déclenche que pour la branche main.
- **Événements** : Il réagit aux envois directs (push) et aux propositions de modifications (pull_request).



Pourquoi ce choix ?

Nous appliquons une stratégie de **Production-Ready Only**. En limitant le pipeline à la branche principale, on économise les ressources de calcul (minutes GitHub Actions) tout en garantissant que rien n'entre en production sans être passé par le "scanner" de tests.

2. Job de Test (La "Quality Gate")

jobs:

test:

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v4

- name: Set up Python 3.13

uses: actions/setup-python@v5

with:

python-version: "3.13"



Analyse Technique :

- **ubuntu-latest** : Utilisation d'un environnement Linux stérile pour garantir la parité avec le

serveur de production (Render/Cloud).

- **Python 3.13** : Alignement strict sur la version locale et celle du Dockerfile pour éviter le bug "ça marche chez moi mais pas ici".

3. Stratégie de Tests E2E (Playwright)

- name: Run E2E Tests

env:

API_KEY: \${ secrets.API_KEY }

run: |

python app.py &

sleep 3

pytest e2e/test_e2e_cold_start.py --reruns 2

Analyse Technique :

- **Injection de Secret** : Utilisation de `${ secrets.API_KEY }` pour sécuriser la clé OpenWeather sans jamais l'exposer.
- **Background Process (&)** : On lance le serveur Flask en arrière-plan pour qu'il soit disponible pendant que les tests tournent.
- **--reruns 2** : Gestion intelligente des tests "flaky" (capricieux) dus aux délais réseaux.

Pourquoi ce choix ?

Contrairement à de simples tests unitaires, les tests **End-to-End (E2E)** vérifient le comportement réel du navigateur. C'est l'ultime rempart pour s'assurer que l'interface "Dark Surface & Gold" s'affiche correctement avant la mise en ligne.

4. Livraison Continue (The Docker Registry)

build-and-push:

needs: [test]

permissions:

packages: write

Analyse Technique :

- **needs: [test]** : C'est la dépendance critique. Si un test échoue, l'image Docker n'est **jamais** construite. On ne livre jamais un produit cassé.
- **Permissions Write** : Autorisation explicite pour que le robot GitHub puisse déposer l'image dans le registre privé (GHCR).

5. Authentification & Publication (GHCR)

- name: Log in to GitHub Container Registry

```
uses: docker/login-action@v3
with:
  registry: ghcr.io
  username: ${{ github.actor }}
  password: ${{ secrets.GITHUB_TOKEN }}
```

Analyse Technique :

- **ghcr.io** : Utilisation du registre de paquets natif de GitHub plutôt que Docker Hub (plus rapide, plus sécurisé car intégré).
- **GITHUB_TOKEN** : Utilisation du jeton automatique temporaire. Pas besoin de gérer des mots de passe manuellement, le système s'auto-authentifie.

Pourquoi ce choix ?

Cela permet de créer une **Single Source of Truth** (Source de vérité unique). Ton image Docker est stockée juste à côté de ton code, prête à être récupérée par tes fichiers **Terraform** pour le déploiement multi-cloud.

6. Métadonnées & Taggage Dynamique

```
- name: Extract metadata
  id: meta
  uses: docker/metadata-action@v5
```

Analyse Technique :

- L'action génère automatiquement des étiquettes (labels) et des tags (ex: main, v1.0, sha-12345).

Pourquoi ce choix ?

En production, il est vital de savoir exactement quelle version du code tourne dans quel conteneur. Ce bloc automatise le versionnage pour éviter toute confusion lors d'un "Rollback" (retour en arrière) nécessaire.

Ce pipeline transforme un dépôt de code en une usine logicielle automatisée, sécurisée et souveraine.